Practical – 1(A1)

Aim: To develop a Java program using an enum and switch expression to simulate a simple vending machine that accepts coins, calculates the total amount inserted, stops when the snack price is reached or exceeded, and displays the remaining change.

Code Screenshot:

A1_vendingmachine.java

```java
import java.util.Scanner;

enum Coin {
    ONE, TWO, FIVE, TEN
}
public class A1_vendingmachine {
    public static void main(String[] args) {
            Scanner sc = new Scanner(System.in);

            int price = 15;
            int total = 0;

            while (total < price) {
                    System.out.print("Enter Coin (ONE/TWO/FIVE/TEN): ");
                    Coin coin = Coin.valueOf(sc.next().toUpperCase());

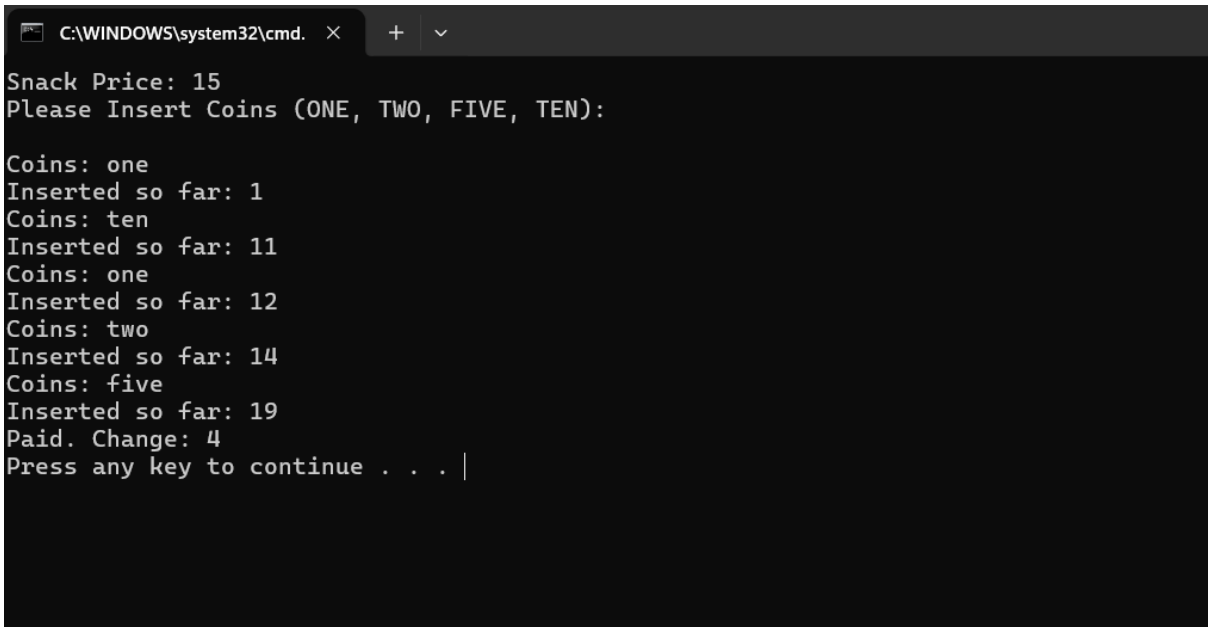
                    int value = 0;

                    switch (coin) {
                        case ONE:
                            value = 1;
                            break;
                        case TWO:
                            value = 2;
                            break;
                        case FIVE:
                            value = 5;
                            break;
                        case TEN:
                            value = 10;
                            break;
    }

                total += value;
                System.out.println("Total = " + total);
            }

            System.out.println("Paid. Change = " + (total - price));
    }
}
```

Output:



Practical – 1(A2)

Aim: To develop a Java program using a record and switch expression to simulate a toll booth system that records vehicle details, calculates the total toll collected, counts different types of vehicles, and identifies the most frequently passing vehicle type.

Code Screenshot:

A2_tollbooth.java

```java
import java.util.Scanner;

public class A2_tollbooth {

    static class VehicleRecord {
        private final String registrationNo;
        private final String category;

        public VehicleRecord(String registrationNo, String category) {
            this.registrationNo = registrationNo;
            this.category = category;
        }

        public String registrationNo() {
            return registrationNo;
        }

        public String category() {
            return category;
        }
    }

    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);
        int totalCollection = 0, motorcycleCount = 0, heavyVehicleCount = 0, passengerCarCount = 0;

        System.out.println("ENTER VEHICLE (Type 'done' for STOP):");

        while (true) {
            System.out.print("Enter Vehicle Number: ");
            String regNum = inputScanner.next().toUpperCase();

            if (regNum.equals("DONE")) {
                break;
            }
```

A2_tollbooth.java

```java
            System.out.print("Vehicle Type (bike, car, truck): ");
            String vehType = inputScanner.next().toLowerCase();

            VehicleRecord currentVehicle = new VehicleRecord(regNum, vehType);

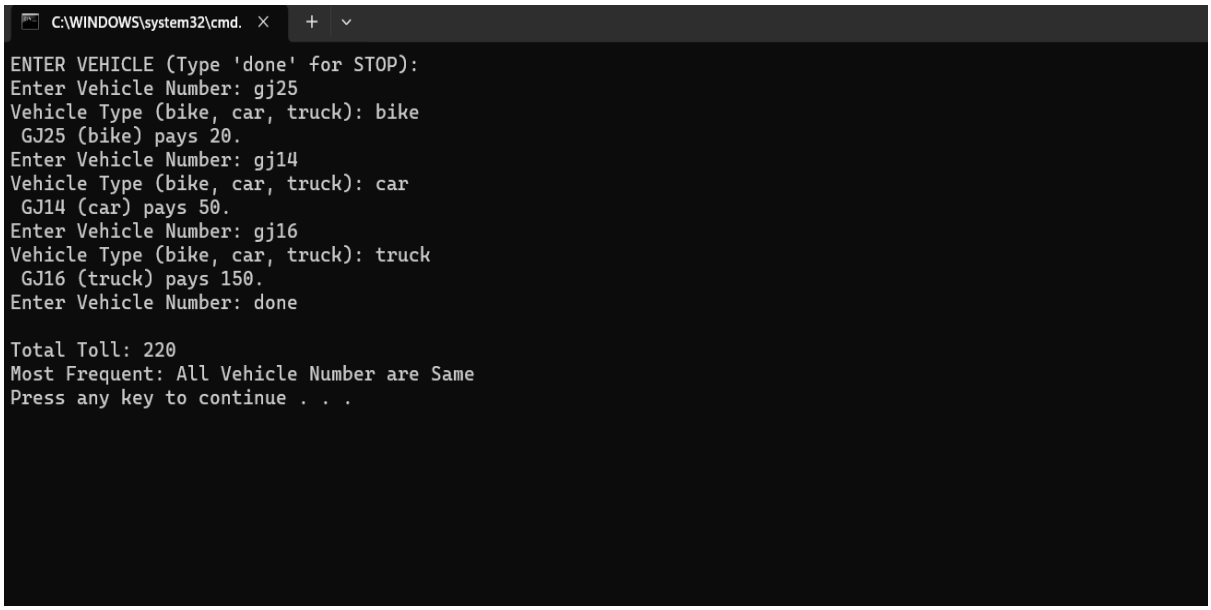
            switch (currentVehicle.category()) {
                case "bike":
                    motorcycleCount++;
                    totalCollection += 20;
                    System.out.println(" " + currentVehicle.registrationNo() + " (" + currentVehicle.category() + ") pays 20.");
                    break;
                case "car":
                    passengerCarCount++;
                    totalCollection += 50;
                    System.out.println(" " + currentVehicle.registrationNo() + " (" + currentVehicle.category() + ") pays 50.");
                    break;
                case "truck":
                    heavyVehicleCount++;
                    totalCollection += 150;
                    System.out.println(" " + currentVehicle.registrationNo() + " (" + currentVehicle.category() + ") pays 150.");
                    break;
                default:
                    break;
            }
        }
```

```
        System.out.println("\nTotal Toll: " + totalCollection);

        String mostFrequentType;
        if (motorcycleCount > passengerCarCount && motorcycleCount > heavyVehicleCount) {
            mostFrequentType = "bike";
        } else if (passengerCarCount > motorcycleCount && passengerCarCount > heavyVehicleCount) {
            mostFrequentType = "car";
        } else if (heavyVehicleCount > motorcycleCount && heavyVehicleCount > passengerCarCount) {
            mostFrequentType = "truck";
        } else if (motorcycleCount == passengerCarCount && motorcycleCount == heavyVehicleCount) {
            mostFrequentType = "All Vehicle Number are Same";
        } else if (motorcycleCount == passengerCarCount) {
            mostFrequentType = "cars & bikes both";
        } else if (motorcycleCount == heavyVehicleCount) {
            mostFrequentType = "bikes & trucks both";
        } else if (passengerCarCount == heavyVehicleCount) {
            mostFrequentType = "cars & trucks both";
        } else {
            mostFrequentType = "Null";
        }

        if (totalCollection == 0) {
            System.out.println("NOT any Vehicle Pass from TOLL.");
        } else {
            System.out.println("Most Frequent: " + mostFrequentType);
        }
    }
}
```

Output:



Practical – 1(A3)

Aim: To develop a Java program using an enum, switch expression, and methods to implement the Rock-Paper-Scissors-Lizard-Spock game, determine the winner of each round, maintain scores over multiple rounds, and declare the overall winner after five rounds.

Code Screenshot:

```java
import java.util.Random;
import java.util.Scanner;

public class A3_RPSLS {

    enum GameMove {
        ROCK, PAPER, SCISSORS, LIZARD, SPOCK
    }

    static int determineWinner(GameMove playerChoice, GameMove opponentChoice) {
        if (playerChoice == opponentChoice) {
            return 0; // draw
        }

        switch (playerChoice) {
            case ROCK:
                return (opponentChoice == GameMove.SCISSORS || opponentChoice == GameMove.LIZARD) ? 1 : -1;
            case PAPER:
                return (opponentChoice == GameMove.ROCK || opponentChoice == GameMove.SPOCK) ? 1 : -1;
            case SCISSORS:
                return (opponentChoice == GameMove.PAPER || opponentChoice == GameMove.LIZARD) ? 1 : -1;
            case LIZARD:
                return (opponentChoice == GameMove.SPOCK || opponentChoice == GameMove.PAPER) ? 1 : -1;
            case SPOCK:
                return (opponentChoice == GameMove.SCISSORS || opponentChoice == GameMove.ROCK) ? 1 : -1;
            default:
                return 0;
        }
    }

    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);
        Random randomGen = new Random();
        int playerScore = 0;
        int compScore = 0;

        System.out.println("===== Rock Paper Scissors Lizard Spock =====");
        System.out.println("Moves: ROCK, PAPER, SCISSORS, LIZARD, SPOCK");
```

```
        for (int roundNo = 1; roundNo <= 5; roundNo++) {
            System.out.println("\nRound " + roundNo);

            // Computer selection
            GameMove compMove = GameMove.values()[randomGen.nextInt(5)];

            // Player selection
            System.out.print("Enter your move: ");
            String rawInput = inputScanner.next().toUpperCase();

            GameMove playerMove;
            try {
                playerMove = GameMove.valueOf(rawInput);
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid Move! Round Skipped.");
                continue;
            }

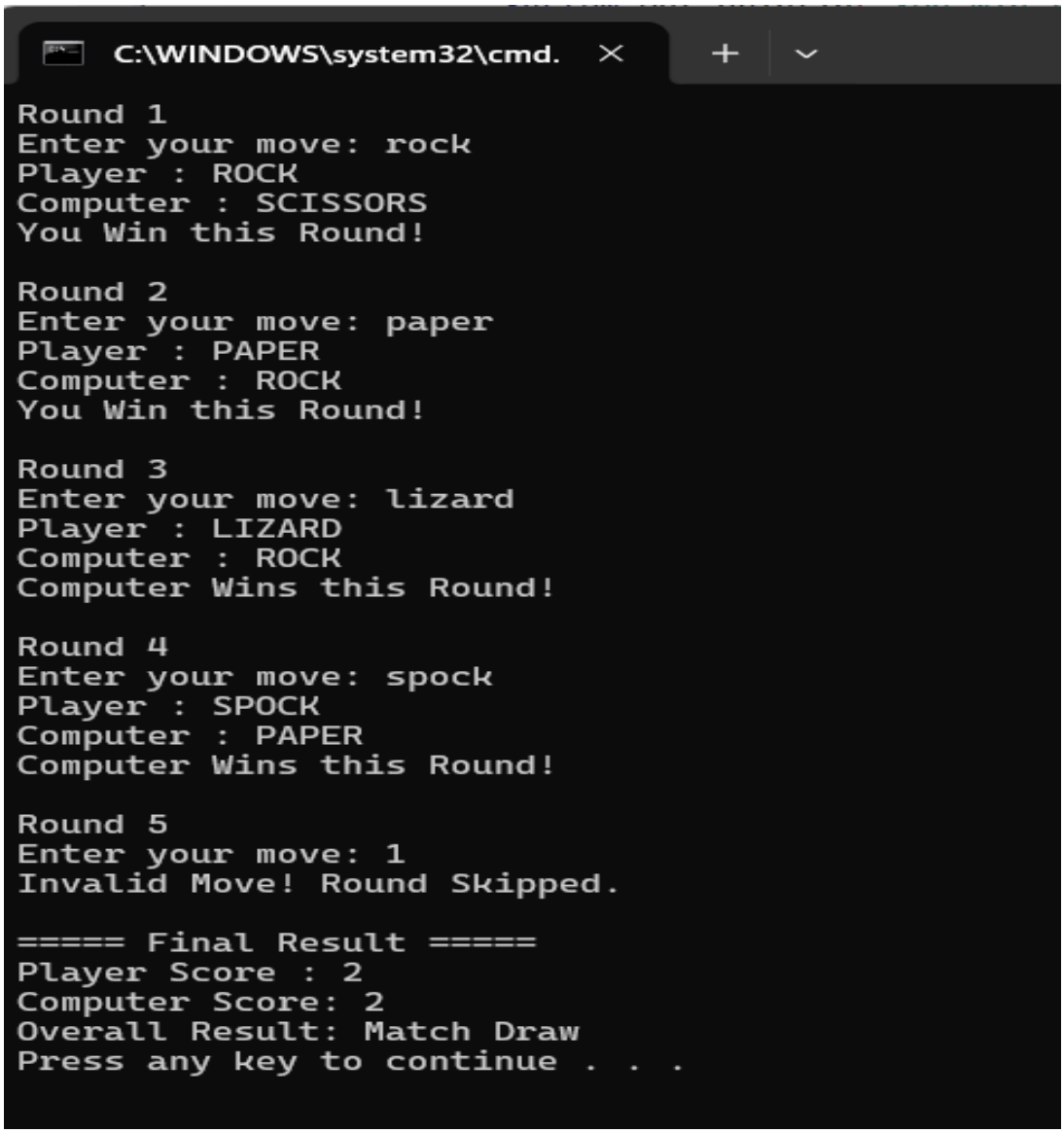
            int roundOutcome = determineWinner(playerMove, compMove);
            System.out.println("Player : " + playerMove);
            System.out.println("Computer : " + compMove);

            if (roundOutcome == 1) {
                System.out.println("You Win this Round!");
                playerScore++;
            } else if (roundOutcome == -1) {
                System.out.println("Computer Wins this Round!");
                compScore++;
            } else {
                System.out.println("Round Draw!");
            }
        }

        System.out.println("\n===== Final Result =====");
        System.out.println("Player Score : " + playerScore);
        System.out.println("Computer Score: " + compScore);

        if (playerScore > compScore) {
            System.out.println("Overall Winner: You win " + playerScore + "-" + compScore);
        } else if (compScore > playerScore) {
            System.out.println("Overall Winner: Computer wins " + compScore + "-" + playerScore);
        } else {
            System.out.println("Overall Result: Match Draw");
        }
    }
}
```

Output:


C:\WINDOWS\system32\cmd.
Round 1
Enter your move: rock
Player : ROCK
Computer : SCISSORS
You Win this Round!
Round 2
Enter your move: paper
Player : PAPER
Computer : ROCK
You Win this Round!
Round 3
Enter your move: lizard
Player : LIZARD
Computer : ROCK
Computer Wins this Round!
Round 4
Enter your move: spock
Player : SPOCK
Computer : PAPER
Computer Wins this Round!
Round 5
Enter your move: 1
Invalid Move! Round Skipped.
===== Final Result =====
Player Score : 2
Computer Score: 2
Overall Result: Match Draw
Press any key to continue . . .

Learning Outcomes:

1. Vending Machine
   - Understand the use of enum to represent fixed constants.
   - Learn to use switch expressions for value mapping.
   - Practice using loops, Scanner, and conditional statements.
   - Develop logic for calculating totals and returning change.

2. Toll Booth
   - Understand the concept and usage of Java Records.
   - Learn to use switch expressions for toll calculation.
   - Practice counting, accumulating totals, and processing user input.
   - Develop logic to determine the most frequently occurring vehicle type.

3. Rock-Paper-Scissors-Lizard-Spock (RPSLS)
   - Understand the use of enum in game development.
   - Learn to implement decision-making using switch expressions.
   - Practice creating and calling methods to determine game results.
   - Gain experience with random number generation, loops, score tracking, and overall game logic.

GitHub Repository:

https://github.com/d26it118-png/OOP_PRACTICALS.git